An Introduction to JavaScript

03 April 2026 19:09

What is Java Script?
JavaScript was initially created to "make web pages alive".
The programs in this language are called scripts. They can be written right in the web pages, HTML and run automatically as the page loads.

Scripts are provided and executed as plain text. They don't need special preparation or compilation to run.

JavaScript is very different from another language called Java.

Why it is called JS?
It initially had another name: **LiveScript**. But Java was very popular at that time, so it was decided that positioning a new language as a younger brother of Java would help.
But as it evolved, JavaScript become a fully independent language with its own specification called **ECMAscript** and now it had no relation to Java at all.

Today, JavaScript can execute not only in browser, but also on the server or actually on any device that has a special program called the JavaScript engine.

The browser has an embedded engine, sometimes called "**JavaScript Virtual machine.**"
Different engines have different code names, for example:

- **V8 - in Chrome, Edge, Opera**
- **SpiderMonkey - in Firefox**
- **… There are other code names like "chakra" for IE "JavascriptCore", "Nitro" and SquirrelFish for Safari Etc.**

The terms above are good to remember because they are used in developer articles on the Internet. We will use them. For instance, if a feature X is supported by V8. Then it probably works in Chrome, Opera and Edge.

How do engine work?
Engines are complicated, but basics are easy.
1. The engine (embedded. If it's a browser) reads (parses) the script.
2. Then it converts (compiles) the script to machine code.
3. And then the machine code runs pretty fast.

Optimizations at each step of the processes. It even watches the compiled

script as it runs, analyze the data that flows through it, and further optimizes the machine code based on that knowledge.

What can in-browser JavaScript do?
Modern JavaScript is a safe programming language. It does not provide low level access to memory or the CPU because it was initially created for browsers which do not require it.

JavaScript's capabilities greatly depend on the environments it's running in. For instance Node.Js supports functions that allow JavaScript to read/write arbitrary files, perform network request, etc.

In browser, JavaScript can do everything related to web page manipulation, interaction with the user and the web server.

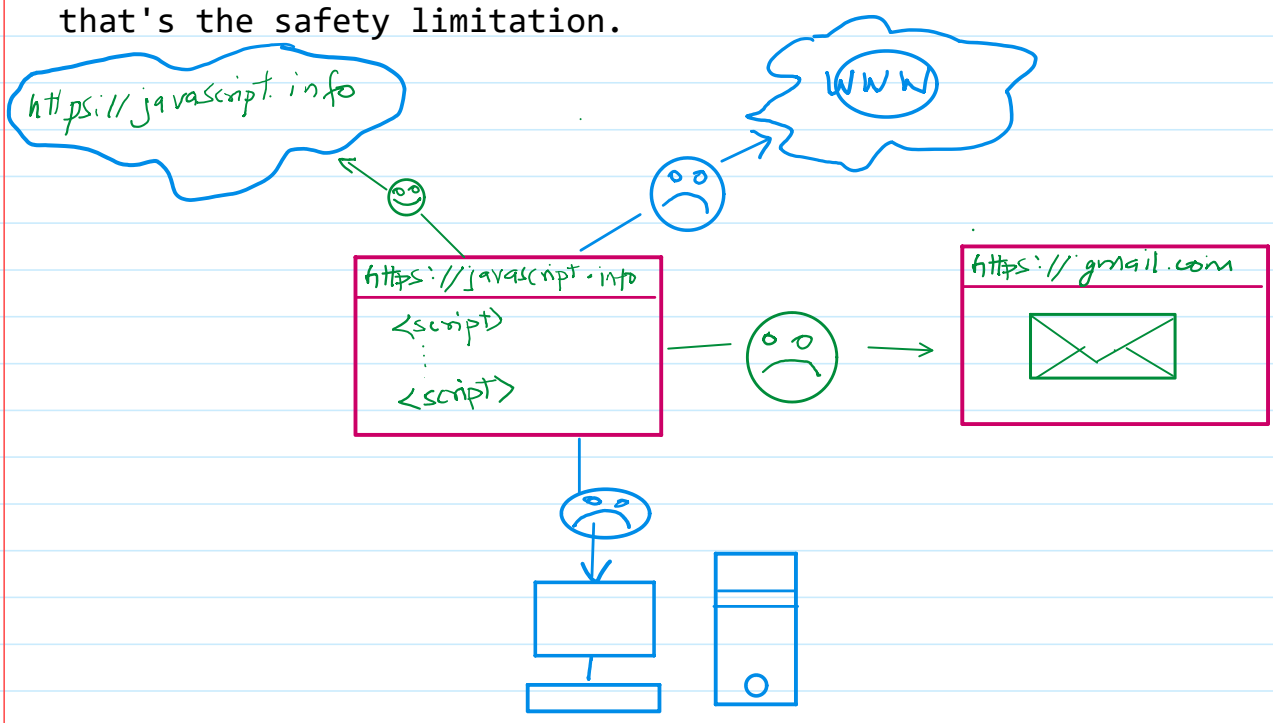
For instance, in browser JavaScript is able to:
1. Add new HTML to the page, change the existing content, modify styles.
2. React to user action. Run on mouse clicks, pointer movements, key presses.
3. Send request over the network to remote server, download and upload files (so called Ajax and comet technologies).
4. Get and set cookies, ask questions to the visitor, show messages.
5. Remember the data on the client side (local storage).

What Can't in-browser JavaScript do?
JavaScript capabilities in the browser are limited to protect the user safety. The aim is to. Went an evil web page from accessing private information or harming the users data.

Examples of such restrictions include:
- JavaScript on a webpage may not read or write arbitrary files on the hard disk. Copy them or execute programs. It has no direct access to OS functions.
- Modern browsers allow it to work with files, but the access is limited. And only provided if the user does certain actions, like dropping a file into a browser window or selecting it via an <input> tag.
- There are ways to interact with the camera or microphone and other devices, but they requires an users explicit permission. So a JavaScript enabled page may not sneakily enable a web camera or observe the surrounding and send the information to the NSA.
- Different tabs or windows generally do not know about each other. Sometimes they do, for example when one window uses JavaScript to open the other one. But even in this case, JavaScript from one page may not access the other page if they come from different sites. (From a different domain, protocol or port)
- This is called the "Same Origin policy". To work around that. Both pages

must agree for data exchange and must contain special Javascript code that handles it.

- This limitation is again for the user's safety. A page from HTTP://anysite.com Which a user has opened must not be able to access another browser tab with the URL http://gmail.com, For example, and steal information from there.
- Javascript can easily communicate over the net to the server where the current page came from. But its ability to receive data from other sides or domain is severely limited. Though possible, it requires explicit agreement (expressed in HTTP headers) From the remote side. Once again, that's the safety limitation.



Such limitations do not exist if JavaScript is used. Outside of the browser. For example on a server. Modern browser also allow plugins or extensions which may ask for extended permissions.

What makes JavaScript unique?

There are at least three great things about JavaScript:

- Full integration with HTML and CSS.
- Simple things are done simply.
- Supported by all major browsers and enabled by default.

Javascript is the. Only browser technology that combines these three things.
That's what make Javascript unique. That's why it is most widespread tool for creating browser interfaces.

That said JavaScript can be used to create servers, mobile applications,

etc.

Languages "over" JavaScript

The syntax of Javascript does not suit everyone's need. Different people want different features.
That's to be expected. Because projects and requirements are different for everyone.
So recently a plethora of new languages appeared. Which are transpired (converted) to JavaScript before they run in browser.

Modern tools make the transpilation very fast and transparent. Actually allowing developers to code in another language and auto converting it. "Under the hood".

Examples of such languages:

- CoffeeScript: Is syntactic sugar for Javascript. It introduces shorter syntax, allowing us to write clearer and more precise code. Usually Ruby Devs like.
- TypeScript: Is concentrated on adding Strict data typing to simplify the development and support of complex system. It is developed by Microsoft.
- Flow: Also add data typing but in different way developed by Facebook.
- Dart: Is a stand-alone language that has its own engine that runs in non-browser environments (like mobile apps) but also can be transpired to Javascript. Developed by Google.
- Brython: Is a Python transpiler to Javascript that enables the writing of application in pure Python without Javascript.
- Kotlin: Is a modern, concise and safe programming language that can target the browser or node.

There are more of course, even if we use one of these transpired languages, we should also know Javascript to really understand what we are doing.